Practice 08 — Design and Defend an AWS Architecture

Objective: in pairs, design an AWS architecture for a real scenario using only the services from weeks 1–7, and **defend** it: say which Well-Architected pillar each part serves and what happens when something fails. This is the skill from today's lecture, and good practice for the midterm.

Timebox: ~45–50 min in class. No AWS account needed. If you're stuck for more than 10 minutes, ask instead of guessing.

> **How you're checked:** this practice is done in class and checked **orally**, as a pair. There is no report. Keep your diagram open (or your paper on the desk). The instructor will ask you to walk through it and will name a failure for you to explain live.

The scenario

NarxozReg is the university's course-registration web app.

- ~**5,000 students**. Most of the year traffic is low, but in **registration week** almost everyone logs in during the first few hours.
- The app must **keep working if one Availability Zone fails**.
- It stores **student data** (names, IDs, chosen courses): it must not be reachable from the internet, and it must be **backed up**.
- Students upload a PDF (e.g. a signed form). Files must be stored durably and **not public**.
- The app servers need to **read and write those files** — without access keys stored on the servers.
- Region: `us-east-1`.

Rules

Use only what we've covered:

- VPC, subnets in **2 AZs**, route tables, Internet Gateway, NAT gateway, security groups (network ACLs optional)
- EC2 (with AMI, instance type, user data), EBS
- S3 (versioning, Block Public Access, lifecycle)
- IAM roles / instance profiles
- RDS (Multi-AZ, automated backups, encryption)
- **Load balancer and Auto Scaling:** draw them as boxes labelled "next week". You don't need details yet, only where they sit.

Definition of done

Be ready to **show** each of these:

- □ A diagram with the nested boxes: **Region** → **VPC** → **2 AZs** → **subnets**, each subnet labelled **public** or **private** with a CIDR.
- □ Every tier (entry point, app servers, database) present in **both AZs** (for RDS: primary and standby).
- □ Arrows for the request flow, labelled with ports (e.g. `HTTPS 443`, `8080`, `5432`).
- □ A **security group table**: each SG, its inbound rule and its **source** (a CIDR or another SG).
- □ Where the student data and the PDFs live, and how each is backed up or protected.
- □ The **design table** (below), filled in for at least 6 components.
- □ Both partners can answer the **failure walk-through** questions.

Task 1 — Draw the architecture (~20 min)

Use **diagrams.net** (draw.io, free, no login) with the AWS icon set: in *More Shapes*, tick the newest **AWS** library on the list, or draw on paper and take a photo.

Start from the skeleton, then add services:

```
Region us-east-1
└─ VPC 10.0.0.0/16  (+ Internet Gateway)
   ├─ AZ a: public subnet | app subnet (private) | DB subnet (private)
   └─ AZ b: public subnet | app subnet (private) | DB subnet (private)
```

Then place:

1. The **entry point** (load balancer, "next week") in the public subnets.
2. **NAT gateway(s)** in the public subnet(s). Decide: one, or one per AZ? Be ready to say why.
3. **App servers** (EC2) in the private app subnets of **both** AZs, each with an **instance profile**.
4. **RDS** in a DB subnet group across both DB subnets, **Multi-AZ**.
5. An **S3 bucket** for the PDFs, outside the VPC box.
6. Arrows and ports for: user → entry point → app → DB, and app → S3.

Task 2 — Security groups (~5 min)

Fill this in for your design:

Security group	Attached to	Inbound port(s)	Source
SG-LB	load balancer		
SG-APP	app servers		
SG-DB	RDS		

Rule of thumb from the lecture: each tier only trusts the tier in front of it, and the source is a **security group**, not a CIDR, wherever you can.

Task 3 — Design table (~10 min)

For at least **6** components, write one line each:

Component	Well-Architected pillar(s)	What failure or risk it handles
App servers in 2 AZs	Reliability	One AZ (or one instance) goes down
RDS Multi-AZ		
Automated RDS backups		
Instance profile (IAM role)		
S3 Block Public Access + versioning		
Private subnets for app and DB		
NAT gateway placement		
Stop/delete idle test resources		

Pillars: operational excellence, security, reliability, performance efficiency, cost optimization, sustainability.

Task 4 — Failure walk-through (with the instructor)

The instructor picks some of these and you explain, using **your** diagram, what happens:

1. **AZ a goes down** completely.
2. The **RDS primary** instance fails.
3. One **app server is terminated** by mistake.
4. A developer **leaks an AWS access key** on GitHub. Which parts of your design limit the damage?
5. In registration week, **traffic is 10× normal**. What breaks first, and which "next week" box fixes it?
6. The monthly bill **doubles** after a test week. Where do you look?

Prepare short answers together before the instructor comes to you.

Stretch (optional) — Estimate the cost

Open the **AWS Pricing Calculator** (`calculator.aws` , no login needed). Region **US East (N. Virginia)**. Add:

- 2 × EC2 instances (pick a small type, e.g. `t3.micro` , on-demand, 24 h/day)

- RDS for PostgreSQL, a small instance class, **Multi-AZ**, 20 GB gp3
- S3 Standard, a few GB
- One NAT gateway (and then two, to compare)

Note the monthly total. Which line is the biggest? What would you change outside registration week?

Track A (optional) — Look at a VPC preview in Learner Lab

Only if your Learner Lab session is running and time allows, per the Learner Lab Readme: open **VPC → Create VPC → "VPC and more"**, set 2 AZs, 2 public and 4 private subnets, and look at the **preview map** on the right. Compare it with your diagram. **Don't click Create**: NAT gateways cost money and you don't need real resources for this practice. Cancel when done.

Be ready to explain

1. Why is the database in a **private** subnet, and how do the app servers still reach it?
2. What does RDS **Multi-AZ** do during a failure, and why doesn't the app need a new endpoint?
3. Why does SG-DB use **SG-APP** as its source instead of the app subnet's CIDR?
4. How do the app servers get permission to write PDFs to S3 without an access key on disk?
5. Where is the single point of failure if you use **one** NAT gateway, and what does fixing it cost?
6. Name one design choice that improves **reliability** but costs more, and one that **saves money** without hurting reliability.

Common mistakes

- **Only one AZ drawn.** "Survive one AZ failure" means every tier appears in both AZs: entry point, app servers, database standby.
- **App servers or database in a public subnet.** Only the entry point (and the NAT gateway) belongs there.
- **SG-DB open to `0.0.0.0/0`**, or to the whole VPC CIDR. Use SG-APP as the source.
- **S3 drawn inside a subnet.** S3 is a regional service outside your VPC; draw it outside the VPC box.
- **Access keys on the servers.** Use an instance profile.
- **No backups.** Multi-AZ is not a backup: a deleted table is deleted on the standby too. You also need automated backups / snapshots.
- **Arrows without ports.** The reviewer can't check your security groups against the diagram.
- **Creating real resources "to check".** This practice needs no AWS; NAT gateways and Multi-AZ RDS bill by the hour.